CPSC-406 Report

Andrew Floyd
Chapman University

February 15, 2026

Abstract

Contents

1 Introduction **1**

2 Week by Week **1**

2.1 Week 1 1

2.1.1 Exercise 1: Word Processing with DFAs 1

2.1.2 Exercise 2: Designing DFAs 3

3 Synthesis **5**

4 Evidence of Participation **5**

5 Conclusion **5**

1 Introduction

2 Week by Week

2.1 Week 1

2.1.1 Exercise 1: Word Processing with DFAs

Notes and Homework:

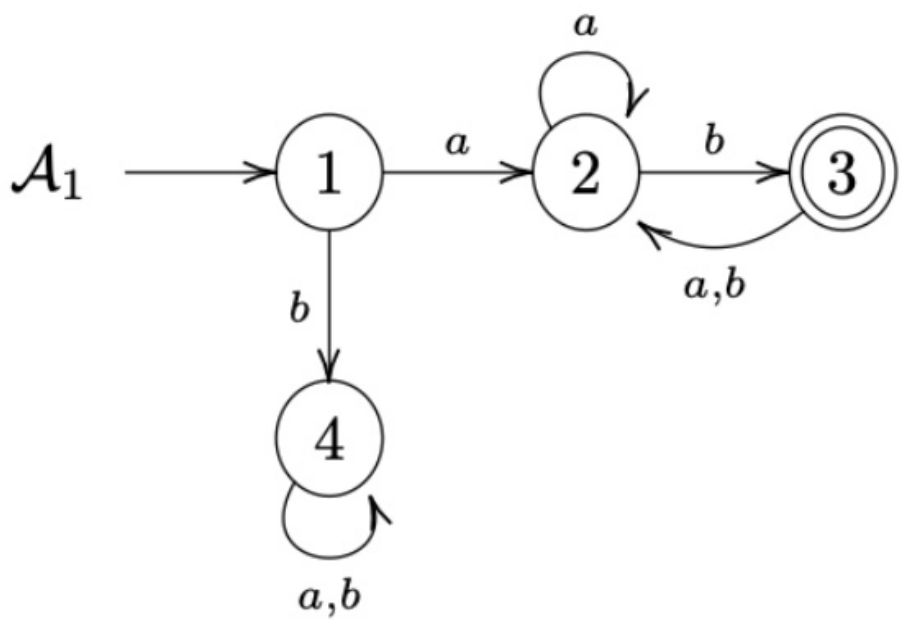

Figure 1: Automata A_1 and A_2

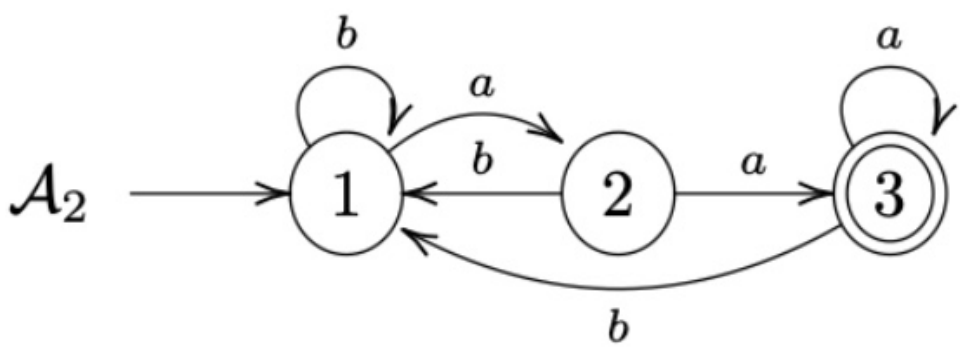

Figure 2: Additional automata reference

Question 1: Which of the following words are accepted/refused by A_1 and A_2, respectively? Complete the table.

w	accepted by A_1?	accepted by A_2?
aaa	no	yes
aab	yes	no
aba	no	no
abb	no	no
baa	no	yes
bab	no	no
bba	no	no
bbb	no	no

Table 1: Acceptance table for A_1 and A_2

Question 2: More generally, can you completely describe the languages $L(A_k)$ accepted by A_k, for $k = 1, 2$?

The language $L(A)$ accepted by this DFA is:

$$L(A) = \{w \in \{a, b\}^* \mid \text{the number of } a\text{'s in } w \text{ is even}\}$$

2.1.2 Exercise 2: Designing DFAs

Design DFAs whose accepted languages are given as follows:

1. All the words that end with ab
2. All the words that contain ab
3. All the words that contain an odd number of a's and an odd number of b's
4. All the words that contain an even number of a's and an odd number of b's
5. All the words such that any three consecutive characters contain at least one a
6. All the words that contain aba

DFA Diagrams:

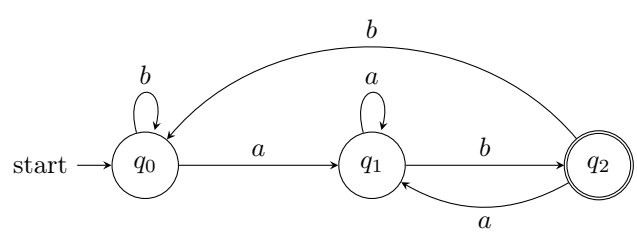

Figure 3: DFA for words ending with ab

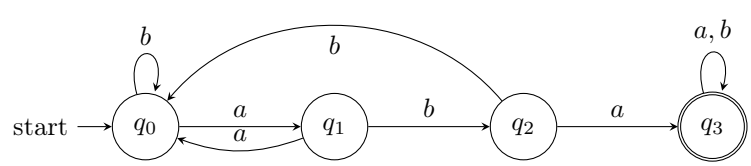

Figure 4: DFA for words containing aba

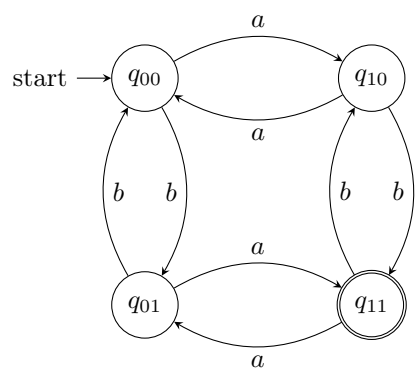

Figure 5: DFA for words with odd number of a's and odd number of b's

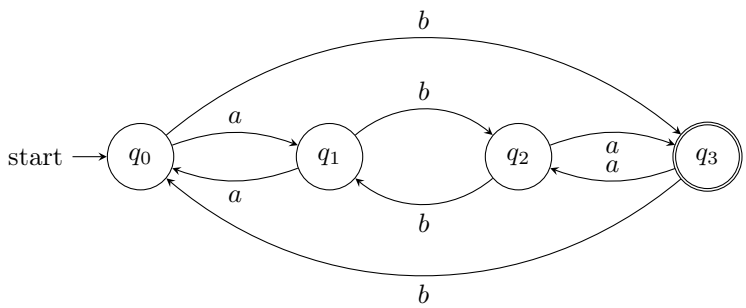

Figure 6: DFA for words with even number of a's and odd number of b's

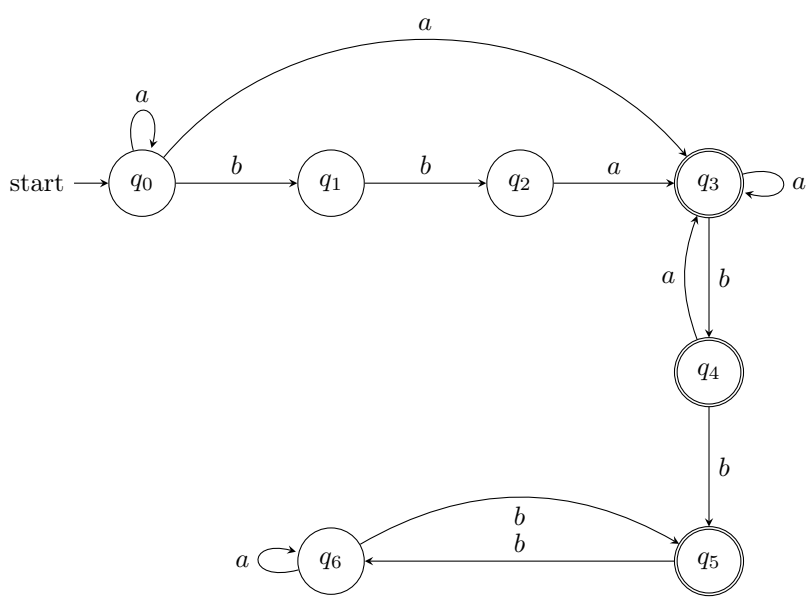

Figure 7: DFA for words where any three consecutive characters contain at least one a

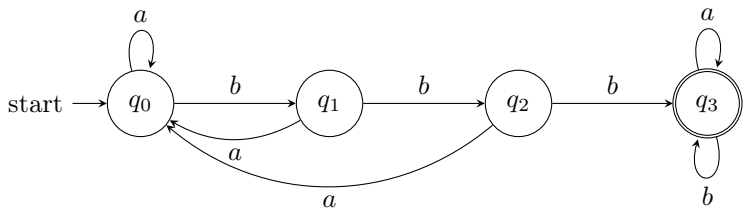

Figure 8: DFA for words containing bbb

What do you notice when comparing the various automata?

When comparing the DFAs designed above, several patterns emerge:

- **State count reflects language complexity:** Simple properties like "ends with ab" require only 3 states, while tracking multiple independent conditions (like counting both a's and b's modulo 2) requires 2^n states where n is the number of independent conditions being tracked.
- **Parity-based languages form grid structures:** DFAs for languages involving "odd/even" counts tend to have states arranged in a grid pattern, where each dimension corresponds to one parity condition. The transitions toggle between states along each axis.
- **Substring containment uses linear chains:** DFAs that check for substring containment (like "contains ab" or "contains bbb") have a linear chain of states tracking progress toward finding the substring, with "reset" transitions back to earlier states when the pattern is broken.
- **Accepting states indicate language structure:** For "contains" languages, once the target is found, we stay in an accepting sink state. For parity languages, specific combinations of parities determine acceptance.
- **Self-loops handle "don't care" inputs:** When an input symbol doesn't affect progress toward acceptance (like a's in "contains bbb" after reaching the accepting state), self-loops keep the automaton in place.

3 Synthesis

4 Evidence of Participation

5 Conclusion

References

[BLA] Author, Title, Publisher, Year.